

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def linear_regression(x, y):
    n = len(x)
    x_mean = np.mean(x)
    y_mean = np.mean(y)
    # Calculate slope (a1)
    numerator = np.sum((x - x_mean) * (y - y_mean))
    denominator = np.sum((x - x_mean) ** 2)
    slope = numerator / denominator
    # Calculate intercept (a0)
    intercept = y_mean - slope * x_mean
    return slope, intercept
```

```
def predict(x, slope, intercept):
    return slope * x + intercept
```

```
def calculate_errors(y, y_pred):
    n = len(y)
    # Residual sum of squares (RSS)
    S_r = np.sum((y - y_pred) ** 2)
    # Mean Squared Error (MSE)
    mse = S_r / n
    # Standard Error of the Estimate (SEE)
    see = np.sqrt(S_r / (n - 2))
    return mse, see
```

```
def linear_interpolation(x0, y0, x1, y1, x):
    return y0 + ((y1 - y0) / (x1 - x0)) * (x - x0)

def calculate_relative_error(true_value, estimated_value):
    return abs((true_value - estimated_value) / true_value) * 100
```

```
def quadratic_interpolation(x_vals, y_vals, x):
    x0, x1, x2 = x_vals
    y0, y1, y2 = y_vals
    b1 = (y1 - y0) / (x1 - x0)
    b2 = ((y2 - y1) / (x2 - x1) - b1) / (x2 - x0)
    # Quadratic interpolation
    fx = y0 + b1 * (x - x0) + b2 * (x - x0) * (x - x1)
    return fx
```

```
def newton_interpolation(x_vals, y_vals, x):
    # Calculate the divided differences table
    n = len(x_vals)
    table = np.zeros((n, n))
    table[:, 0] = y_vals # First column is y-values

    for j in range(1, n):
        for i in range(n - j):
            table[i, j] = (table[i + 1, j - 1] - table[i, j - 1]) /

    # Extract the coefficients (first row of the divided difference
    coefficients = table[0, :]

    # Compute the Newton interpolating polynomial
    result = coefficients[0]
    product_terms = 1.0
    for i in range(1, n):
        product_terms *= (x - x_vals[i - 1])
        result += coefficients[i] * product_terms
    return result
```

```
def lagrange_interpolation(x_vals, y_vals, x):
    n = len(x_vals)
    result = 0
    for i in range(n):
        # Compute the Lagrange basis polynomial L_i(x)
        L_i = 1
        for j in range(n):
            if i != j:
                L_i *= (x - x_vals[j]) / (x_vals[i] - x_vals[j])
        # Add the contribution of f(x_i) * L_i(x)
        result += y_vals[i] * L_i
    return result
```

Task

```
# ข้อมูลจากโจทย์ Tasks
time = np.array([6, 8, 10, 12, 14, 16, 18], dtype=float)
pm25 = np.array([60, 160, 100, 140, 120, 130, 160], dtype=float)

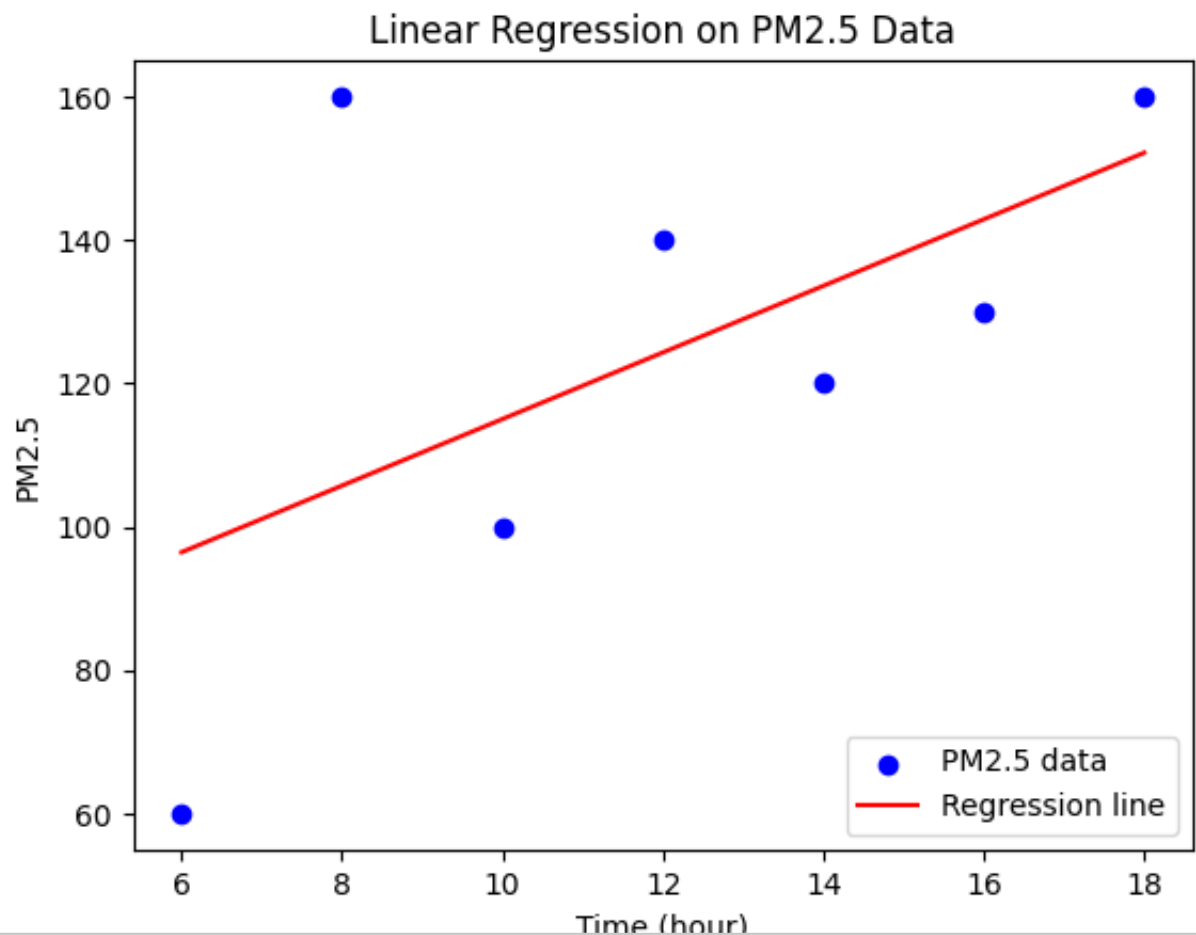
x_targets = np.array([7, 11], dtype=float) # เวลาที่ต้องการทำนาย
```

```
slope, intercept = linear_regression(time, pm25)
pm25_lr = predict(x_targets, slope, intercept)

print(f"Slope (a1): {slope}")
print(f"Intercept (a0): {intercept}")
print(f"Linear Regression PM2.5 at 7 AM, 11 AM: {pm25_lr}")
```

```
Slope (a1): 4.642857142857144
Intercept (a0): 68.57142857142856
Linear Regression PM2.5 at 7 AM, 11 AM: [101.07142857 119.64285714]
```

```
plt.scatter(time, pm25, color='blue', label='PM2.5 data')
plt.plot(np.sort(time), predict(np.sort(time), slope, intercept), c
plt.xlabel('Time (hour)')
plt.ylabel('PM2.5')
plt.title('Linear Regression on PM2.5 Data')
plt.legend()
plt.show()
```



```
# 7 AM อยู่ระหว่าง 6 และ 8
pm25_li_7 = linear_interpolation(6, 60, 8, 160, 7)

# 11 AM อยู่ระหว่าง 10 และ 12
pm25_li_11 = linear_interpolation(10, 100, 12, 140, 11)

print(f"Linear Interpolation PM2.5 at 7 AM: {pm25_li_7}")
print(f"Linear Interpolation PM2.5 at 11 AM: {pm25_li_11}")
```

```
Linear Interpolation PM2.5 at 7 AM: 110.0
Linear Interpolation PM2.5 at 11 AM: 120.0
```

```
#เลือก 3 จุดสำหรับ quadratic interpolation
#สำหรับ 7 AM ใช้ (6, 8, 10)
x_vals_7 = [6, 8, 10]
y_vals_7 = [60, 160, 100]
pm25_qi_7 = quadratic_interpolation(x_vals_7, y_vals_7, 7)

#สำหรับ 11 AM ใช้ (8, 10, 12)
x_vals_11 = [8, 10, 12]
y_vals_11 = [160, 100, 140]
pm25_qi_11 = quadratic_interpolation(x_vals_11, y_vals_11, 11)

print(f"Quadratic Interpolation (6,8,10) PM2.5 at 7 AM: {pm25_qi_7}")
print(f"Quadratic Interpolation (8,10,12) PM2.5 at 11 AM: {pm25_qi_11}")
```

```
Quadratic Interpolation (6,8,10) PM2.5 at 7 AM: 130.0
Quadratic Interpolation (8,10,12) PM2.5 at 11 AM: 107.5
```

```
results = {
    "Method": ["Linear Regression", "Linear Interpolation", "Quadratic Interpolation"],
    "PM2.5 @ 7 AM": [pm25_lr[0], pm25_li_7, pm25_qi_7],
    "PM2.5 @ 11 AM": [pm25_lr[1], pm25_li_11, pm25_qi_11],
}
results
```

```
{'Method': ['Linear Regression',
'Linear Interpolation',
'Quadratic Interpolation'],
'PM2.5 @ 7 AM': [np.float64(101.07142857142856), 110.0, 130.0],
'PM2.5 @ 11 AM': [np.float64(119.64285714285714), 120.0, 107.5]}
```

ผลการคำนวณค่า PM 2.5 ที่เวลา 7 นาฬิกา และ 11 นาฬิกา จากทั้งสามวิธีมีค่าแตกต่างกัน เนื่องจากหลักการของแต่ละวิธีไม่เหมือนกัน โดยวิธี Linear Regression ใช้ข้อมูลทุกช่วงเวลามาสร้างสมการเส้นตรงเดียว จึงสะท้อนแนวโน้มโดยรวมของข้อมูลทั้งหมด ขณะที่ Linear Interpolation ใช้เพียงข้อมูลสองจุดที่อยู่ใกล้เวลาที่ต้องการคำนวณ ทำให้ค่าที่ได้เป็นการประมาณเฉพาะช่วงเวลาใกล้เคียง ส่วน Quadratic Interpolation ใช้ข้อมูลสามจุดและสร้างเส้นโค้งพหุนาม ทำให้สามารถสะท้อนความโค้งของข้อมูลได้ดีกว่า แต่ผลลัพธ์จะขึ้นอยู่กับ การเลือกจุดข้อมูลทั้งสามจุด ดังนั้นความแตกต่างของค่าที่ได้จึงเกิดจากจำนวนข้อมูลที่ใช้และสมมติฐานของแต่ละวิธีในการประมาณค่า